

S04 / S05. Inventário de dados e plano de ação técnica

Relatório técnico | Foco: o que dá pra fazer com os dados atuais e como evoluir

1. Panorama de dados

O NLP Engine consome `gold_corporativo_ia.corporativo.tb_gold_mov_exame` (75M exames) como fonte única. Cada especialidade aplica filtros na ingestão (tipo de procedimento, janela temporal e palavra-chave de órgão), resultando em ~4,6M laudos processados somados pelas 9 especialidades (com sobreposição entre elas).

A pirâmide do volume efetivo:

Camada	Volume	Natureza
Gold corporativo (todas entidades)	~250M+	Base corporativa FHIR-like
Gold corporativo IA (exames)	75M	Fonte única dos notebooks
Laudos processados pelas 9 especialidades	~4,6M	Após filtros de cada engine
Laudos flagged como relevantes	~1M	Pseudo-labels gerados pelo engine
Laudos homologados (gold standard)	~4K	Validação humana registrada

Taxa de homologação por especialidade

Especialidade	Processados	Homologados	Taxa	Natureza
Hepatologia	1,32M	3.312	0,25%	Pré-triagem médica
DII	104K	443	0,42%	Validação interna do time
Biliar	~200K	209	0,10%	Validação interna do time
Neuroimunologia	~150K	108	0,07%	Validação interna do time
Reumatologia	547K	143	0,026%	Validação interna do time
Colon	1,34M	101	0,008%	Validação interna do time
Birads, Tirads, Pirads, Pulmão	sem dado	não identificado	sem dado	—

Média ponderada (6 especialidades com homologação conhecida): **~0,12%**. O volume absoluto por especialidade (100 a 3.000 casos) é o **gargalo técnico** da evolução do modelo. *Pré-triagem médica* (hepato) refere-se à validação feita pelo médico antes do envio para as enfermeiras navegadoras; *validação interna do time* refere-se a checagens feitas majoritariamente pela Carol antes do deploy, distinta de validação clínica em uso real. Além disso, há volume desconhecido de validações eventuais de navegadoras dispersas em planilhas no OneDrive, recuperáveis via varredura programática (não contabilizadas acima).

2. Variáveis: usadas vs disponíveis

Os notebooks das 9 especialidades consomem apenas o texto do laudo (`laudoExameOriginal`) como feature efetiva. Outras colunas presentes na tabela de entrada (idade, sexo, plaquetas, convênio, regional) são coletadas mas descartadas no SELECT que alimenta o modelo. Há também dados clínicos estruturados no lake que os notebooks não leem.

Grupo	Exemplos	Status no modelo
Usada hoje	<code>laudoExameOriginal</code> (texto)	Única feature efetiva
Coletada, não usada	<code>num_idade_paciente</code> , <code>gen_sexo_paciente</code> , <code>vl_plt</code> , <code>num_dif_tempo_imagem_plt</code> , <code>nome_convênio</code> , <code>emp_regional_unidade</code>	Na tabela de entrada, não entra no modelo
Disponível no lake	<code>tbl_gold_fib_score</code> (FIB-4), <code>tbl_gold_mov_condition</code> (CIDs), <code>tbl_condicoes_saude</code> (42 categorias), <code>silver_tasy_prescr_medica</code>	Notebooks não leem; exige join por <code>id_paciente</code>

3. Estratégias técnicas. Como fazer

Com ~100 a 3.000 laudos homologados por especialidade, treino supervisionado robusto é improvável no volume atual, salvo modelos simples com forte regularização e validação cuidadosa. As estratégias abaixo estão ordenadas por viabilidade: das mais informativas no volume atual para as que dependem de aumento de homologação.

Abordagem de execução: cada estratégia é testada primeiro em notebook exploratório, onde os dados indicam se a técnica agrega valor. Só o que provar ganho objetivo é consolidado em módulo `.py` reutilizável. Evita escalar código de técnica descartada.

Premissa crítica: qualquer uso de variáveis estruturadas via join por `id_paciente` exige filtro temporal estrito (`data da feature ≤ data do laudo`) para evitar *leakage*, ou seja, uso de informação posterior ao laudo para prever o próprio laudo. Atenção também a atrasos de cadastro (feature existia no momento, mas foi registrada depois). Sem essa checagem, o modelo performa em backtest e falha em produção.

3.1. Weight of Evidence (WoE) e Information Value

Triagem exploratória do poder preditivo de cada variável candidata (idade, sexo, PLT, FIB-4, regional, convênio) sem treinar modelo. WoE é interpretável, funciona com poucos dados e produz uma métrica única (IV) por variável. **Pré-requisito:** auditoria descritiva antes do cálculo, ou seja, *missingness* (FIB-4 só existe para quem tem TGO, TGP e PLT medidos), cardinalidade, estabilidade temporal e cobertura por especialidade. Variável com >50% de missing não entra na análise sem tratamento prévio.

Fórmula:

$$\text{WoE}_i = \ln((\% \text{ positivos no bin}_i) / (\% \text{ negativos no bin}_i))$$
$$\text{IV} = \sum (\% \text{ pos}_i - \% \text{ neg}_i) \times \text{WoE}_i$$

Interpretação do IV: <0,02 sem poder; 0,02–0,1 fraco; 0,1–0,3 médio; >0,3 forte. **Pipeline proposto:** (1) binarizar alvo (fl_relevante 0/1 na amostra homologada); (2) discretizar cada variável contínua em 5–10 bins; (3) calcular WoE por bin e IV total; (4) selecionar variáveis com IV > 0,1 para as próximas etapas; (5) descartar ou fazer feature engineering nas demais.

Saída: tabela com IV por variável × especialidade. Resposta direta à pergunta "idade e sexo têm poder preditivo?" sem precisar treinar modelo.

3.2. Backtest de features na amostra homologada

Para variáveis com IV promissor, testar ganho real sobre o baseline atual **sem quebrar produção**. Rodar o engine atual na amostra homologada, registrar previsões, aplicar regra composta (engine + cutoff de feature nova ex: PLT < 150 → prior para fibrose) e comparar com o baseline. **Separação obrigatória:** a amostra usada para seleção de variáveis via IV (seção 3.1) não pode ser a mesma usada no backtest. Aplicar split temporal (IV em período A, validação em período B) ou k-fold. Caso contrário, induz otimismo por dupla função da base. **Teste estatístico:** McNemar para pares (cada laudo tem previsão do baseline e da regra composta), como componente do critério, não como eixo único. **Critério de aceite:** p < 0,05 em McNemar, delta F-beta(β=2) positivo sobre baseline, análise de calibração e de custo clínico por especialidade (β=2 privilegia recall; ajustar por domínio se o custo de FP dominar). O baseline por especialidade sai como subproduto natural deste exercício, sem necessidade de tarefa separada. Se passar, vira proposta de incorporação ao engine.

3.3. Evolução dos embeddings

O Grupo 1b usa hoje paraphrase-multilingual-MiniLM-L12-v2 (multilíngue genérico). Alternativa: pucpr/bioBERTpt-all, BERT pré-treinado em textos biomédicos em português. **Não é substituição trivial:** muda distribuição de scores, exige recalibração de thresholds, recomputação de cache e avaliação em vocabulário fora do domínio biomédico. **Avaliação:** rodar ambos em paralelo na amostra homologada e comparar F-beta(2) e estabilidade.

3.4. Fine-tuning leve (condicional a volume)

Com o volume atual (~100 a 3.000 homologados por especialidade), fine-tuning de encoder completo não é viável. Viável apenas fine-tuning da **última camada** (head) em cima de encoder congelado (BERTimbau ou BioBERTpt), com gradual unfreezing, k-fold (k=5), early stopping, regularização forte. **Ordens de grandeza de referência:** cerca de 1K homologados por especialidade para fine-tuning de head; cerca de 5K para fine-tuning parcial do encoder. Heurísticas de planejamento, sujeitas a revisão conforme prevalência e estabilidade do target.

3.5. Weak supervision (Snorkel)

Usar os engines atuais (regras, embeddings, regex) como *labeling functions*. Snorkel gera pseudo-labels em escala, complementar à base homologada. Útil como entrada para fine-tuning quando o volume subir. **Dependência:** mais informativa depois que as LFs estiverem caracterizadas (3.1 e 3.2).

3.6. Governança de homologação (variável resposta)

Primeira camada de governança: como gerar e manter o gold standard de forma sistemática. Os dados homologados existem mas estão dispersos em fontes distintas, com naturezas diferentes. Antes do fluxo recorrente, é pré-requisito consolidar tudo em uma tabela canônica no Lake.

Consolidação inicial. Criar tabela canônica no Lake (id_predicao, id_exame, id_paciente, especialidade, label TP/FP/FN, fonte, validador, data, observação) e popular com o que existe nas fontes mapeadas:

Fonte	Conteúdo	Volume	Status
Lake (hepato)	Tabela tb_diamond_mod_hepatologia_retorno. Pré-triagem médica antes do envio às navegadoras	3.312 casos	Acessível, já no Lake
OneDrive (time)	Planilhas dispersas com homologações internas (DII, Biliar, Reumato, Neuro, Colon, Pulmão), feitas majoritariamente pela Carol antes do deploy	1.004 conhecidas	Acessível
OneDrive (envio)	Pastas com várias planilhas de envio, algumas potencialmente preenchidas por navegadoras. Não há indexação que permita saber quais; recuperação requer varredura programática	a extrair	Acessível, varredura pendente
Salesforce	Homologações registradas pelas navegadoras e por Carol no contexto operacional do CRM	a confirmar	Sem acesso atual

Mapeamento campo a campo até o schema canônico, mais descoberta da chave de junção com o diamond (id_exame quando disponível, ou paciente + data + procedimento). Trabalho mecânico, delegável. Sem ele, todas as estratégias técnicas (3.1 a 3.5) ficam sem dado real para rodar.

Fluxo recorrente. A cada nova versão do engine (vinculado ao 3.7), uma lib parametrizada (id_projeto, n_positivos, n_negativos) lê as saídas no diamond e gera amostra estratificada (com opção de borderline por score intermediário). Vai pra Carol em formato consumível (planilha ou view). Validação humana e manual; o que automatiza é a geração e a entrega. Retorno grava na tabela canônica, alimentando o gold de forma incremental. **Fase 1:** chamada manual da lib

quando há mudança no engine. **Fase 2:** trigger automático vinculado ao versionamento. Interface dedicada fica como oportunidade futura, condicional ao TAB.

Volume e ritmo. O crescimento do gold passa a ser função da cadência de mudanças e da disponibilidade da Carol. Como referência indicativa, 1K homologados por especialidade viabiliza regressão logística com poucas features e 5K abre caminho para fine-tuning mais robusto. **Protocolo de anotação:** a qualidade do gold depende de guideline formal (definição de TP, FP, FN, casos ambíguos, revisão de discordâncias). Sem isso, aumentar homologação escala ruído.

3.7. Governança de mudança do engine

Segunda camada de governança: como alterar o engine em produção de forma controlada, sem quebrar o que já funciona. Fluxo proposto para qualquer alteração de regra, feature ou modelo:

1. Solicitação negócio	Demanda clínica ou operacional formalizada (médico, navegador, produto)
2. Aprovação técnica	DS avalia viabilidade: há dado para suportar? cabe no engine atual? risco aceitável?
3. Teste	Implementação em branch. Validação em ambiente de dev com amostra homologada
4. Paridade baseline	Comparação objetiva com versão atual: precision, recall, F-beta, McNemar. Nova versão não pode ser pior
5. Validação	Revisão clínica de casos selecionados pelo validador (Carol ou equivalente)
6. Versionamento	Merge com tag semântica (ex: hepato_v4.2), changelog e registro da decisão

Implementação: template de PR em YAML com campos obrigatórios (motivação, mudança, métricas antes/depois, validador clínico, reviewer técnico). Merge só com as duas aprovações. Git tag obrigatória para rastreabilidade. Rollback automatizado se monitoramento detectar degradação fora do IC esperado.

4. Plano de ação. Tarefas para execução

Tarefas no nível de story pronta para o cientista pegar e executar. **Filosofia:** notebook exploratório primeiro (os dados decidem se a técnica agrega), consolidação em módulo .py só do que provar valor. T0 e T1 são bloqueantes: trazem o dado pro Lake e estabelecem o fluxo de homologação que alimenta o resto. T2 a T5 geram aprendizado incremental em paralelo. T6 fecha o ciclo com o workflow de aprovação de mudança.

#	Tarefa	Passos concretos	Entregável
T0	Centralizar todos os homologados no Lake	1) Mapear conteúdo das 4 fontes (Lake hepato, OneDrive time, OneDrive envio, Salesforce). 2) Definir schema canônico da tabela no Lake. 3) Implementar ETL para cada fonte acessível. 4) Construir varredura programática para extrair planilhas potencialmente preenchidas em OneDrive (envio). 5) Solicitar liberação de acesso ao Salesforce; integrar quando disponível. 6) Validar volumes e completude por especialidade.	Tabela canônica de homologados populada no Lake, pronta para servir como input das demais tarefas
T1	Lib parametrizada de geração de amostra	1) Criar lib reutilizável recebendo id_projeto, n_positivos, n_negativos e opção de borderline. 2) Lê as saídas do engine no diamond e aplica amostragem estratificada. 3) Entrega lote para Carol em formato consumível (planilha ou view). 4) Captura retorno e grava na tabela canônica (T0). 5) Fase 1: chamada manual quando há mudança. Fase 2: trigger automático futuro.	Lib .py reutilizável por todas as especialidades, primeira rodada de validação executada com Carol
T2	WoE e IV para variáveis candidatas em hepato	1) Auditoria descritiva: missingness, cardinalidade, distribuição por classe. 2) Definir target binário na amostra homologada. 3) Discretizar contínuas em 5 a 10 bins (quantis). 4) Calcular WoE por bin e IV total. 5) Split temporal para validar.	Notebook exploratório com tabela de IV por variável. Se IV > 0,1, técnica vira módulo .py (woe.py) para reuso em T4
T3	Backtest das variáveis top 3 em hepato (com baseline)	1) Fixar split temporal distinto de T2. 2) Gerar predição baseline e predição com regra composta (engine + cutoff de feature). 3) McNemar pareado. 4) F-beta por domínio. 5) Análise por fatias (idade, regional). Baseline por especialidade sai como subproduto.	Notebook exploratório com tabela comparativa baseline vs engine+feature, parecer go/no-go por variável. Módulo .py só se validado
T4	Replicar T2 e T3 em DII, Biliar, Reumato, Neuro	1) Aplicar módulos .py consolidados em T2/T3. 2) Ajustar target por especialidade. 3) Documentar prevalência de positivos por amostra.	4 notebooks análogos aos de hepato usando módulos comuns, com sumário comparativo entre especialidades
T5	Avaliar BioBERTpt vs MiniLM em 1 especialidade	1) Clonar engine atual. 2) Substituir modelo em ponto único (encoder). 3) Recomputar embeddings de seeds e cache. 4) Recalibrar thresholds no split de validação. 5) Rodar ambos na amostra homologada e comparar.	Notebook de benchmark com F-beta, recall, latência, memória. Se ganho confirmado, substituição vira patch via fluxo de governança (3.7)
T6	Template de PR + workflow de aprovação de mudança	1) Definir YAML template (motivação, mudança, métricas antes/depois, validador). 2) Configurar CI com gate de paridade baseline. 3) Documentar procedimento de rollback.	Template YAML no repositório, pipeline CI configurado, runbook de rollback

5. Síntese

Volume de homologados por especialidade (100 a 3.000 casos, ~0,12% do total processado) é fator determinante do que é viável. **T0 e T1 são bloqueantes:** sem centralizar os homologados das 4 fontes no Lake (T0) e estabelecer o fluxo recorrente de geração de amostra (T1), as demais tarefas ficam sem dado real para executar. WoE/IV e backtest com validação fora da amostra de seleção são os passos mais informativos no volume atual. Cada etapa em notebook; só o que provar valor vira módulo .py consolidado. **Governança em duas camadas:** homologação (3.6) e mudança do engine (3.7).